DESENVOLVIMENTO DE APLICATIVOS MOBILE · MÓDULO 09

Firebase: Backend como Serviço

Authentication, Cloud Firestore, regras de segurança, Storage, Cloud Messaging, Crashlytics e alternativas

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Explicar o modelo *Backend as a Service* e os principais serviços do Firebase.
- Configurar um projeto Firebase em um app Android.
- Autenticar usuários com e-mail/senha e conta Google.
- Modelar, gravar e observar dados em tempo real no Cloud Firestore.
- Proteger os dados com regras de segurança e testá-las no emulador local.
- Enviar arquivos ao Storage, receber notificações push e monitorar falhas.

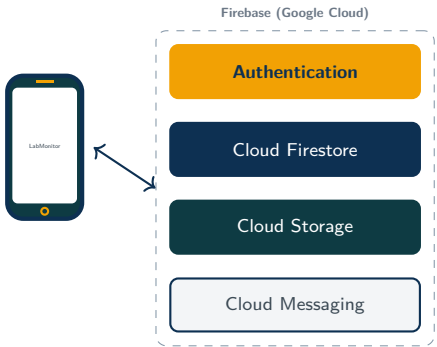
Roteiro

Neste módulo

- Visão geral
- Configuração
- Authentication
- Cloud Firestore
- Regras de segurança
- Storage, mensagens e monitoramento
- Fechamento

Visão geral

Backend as a Service



O **Firebase** oferece, como serviços gerenciados, as peças que todo app conectado precisa:

- login de usuários;
- banco de dados em tempo real com cache offline;
- armazenamento de arquivos;
- notificações push;
- relatórios de falhas e métricas de uso.

Dica

Ótimo para protótipos, TCCs e apps pequenos e médios: você se concentra no app e não em servidores.

Serviços e planos

Serviço	Para quê
Authentication	login (e-mail, Google, telefone, anônimo...)
Cloud Firestore	banco NoSQL de documentos, tempo real
Cloud Storage	fotos, PDFs, arquivos grandes
Cloud Messaging (FCM)	notificações push
Cloud Functions	código no servidor disparado por eventos
Crashlytics	relatórios de falhas em produção
Analytics	eventos e funis de uso
Remote Config	mudar parâmetros sem publicar versão
App Check	garante que só o seu app acessa o backend
Firebase AI Logic	acesso a modelos Gemini a partir do app (Módulo 14)

Planos

Spark: gratuito, sem cartão; ao atingir a cota, o serviço para até o próximo período.
Blaze: pague pelo uso, com as mesmas cotas gratuitas.

Cloud Storage exige Blaze

Desde **03/02/2026**, o Cloud Storage para Firebase só funciona em projetos no plano Blaze (a cota gratuita continua valendo). Ao ativar o Blaze, configure um **alerta de orçamento**.

Configuração

Adicionando o Firebase ao app

1. Crie um projeto no console do Firebase.
2. Adicione um app Android com o `applicationId` do app.
3. Informe o SHA-1 de debug (`./gradlew signingReport`), exigido pelo login Google.
4. Baixe o `google-services.json` para `app/`.
5. Adicione o plugin e as bibliotecas.

Dica

Tools → **Firebase** no Android Studio automatiza esses passos.

app/build.gradle.kts

```
plugins {
  alias(libs.plugins.google.services)
  alias(libs.plugins.crashlytics)
}
dependencies {
  // o BoM alinha as versões de tudo
  implementation(platform(libs.firebase.bom))
  implementation(libs.firebase.auth)
  implementation(libs.firebase.firestore)
  implementation(libs.firebase.storage)
  implementation(libs.firebase.messaging)
  implementation(libs.firebase.crashlytics)
}
```

Atenção

Os antigos módulos `-ktx` foram descontinuados: as APIs Kotlin (`Firebase.auth`, `snapshots()`...) agora estão nos módulos principais.

Authentication

Login com e-mail e senha

AuthRepository.kt

```
class AuthRepository @Inject constructor() {
  private val auth = Firebase.auth

  // usuário atual como Flow (null = deslogado)
  val usuario: Flow<FirebaseUser?> = callbackFlow {
    val l = FirebaseAuth.AuthStateListener {
      trySend(it.currentUser)
    }
    auth.addAuthStateListener(l)
    awaitClose { auth.removeAuthStateListener(l) }
  }

  suspend fun entrar(email: String, senha: String) =
      runCatching {
    auth.signInWithEmailAndPassword(email, senha).await()
  }
  suspend fun cadastrar(email: String, senha: String) =
      runCatching {
    auth.createUserWithEmailAndPassword(email, senha).await()
  }
  fun sair() = auth.signOut()
}
```

await()

As APIs do Firebase devolvem `Task`. A extensão `await()` (de `kotlinx-coroutines-play-services`) as transforma em chamadas `suspend`.

Navegação condicionada

`usuario` nulo leva ao login; senão, ao painel. A sessão persiste entre aberturas.

Atenção

O app nunca guarda a senha.

Login com Google via Credential Manager



Credential Manager

Unifica senhas, *passkeys* e login Google numa única folha de seleção (substitui o antigo *Google Sign-In*).

Passos

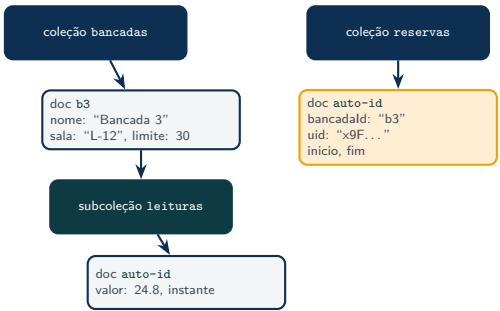
1. Ative o provedor Google no console.
2. Pegue o *Web client ID* do projeto.
3. Peça uma credencial `GetGoogleIdOption` ao `CredentialManager`.
4. Troque o ID token por uma credencial Firebase.

Atenção

Cadastre o SHA-1 de cada chave que assina o app: debug, upload e Play App Signing (Módulo 12).

Cloud Firestore

Modelo de dados: coleções e documentos



- **Documento**: conjunto de campos (como um JSON), até 1 MiB.
- **Coleção**: conjunto de documentos.
- Documentos podem ter **subcoleções**.
- Não há *joins*: modele pensando nas **consultas** que a tela fará.

Dica

É comum **duplicar** dados (ex.: o nome da bancada dentro da reserva) para evitar leituras extras. O custo é cobrado por documento lido.

Gravando e lendo em tempo real

ReservaRepository.kt

```
data class ReservaDoc(          // construtor vazio p/ Firestore
  val bancadaId: String = "", val uid: String = "",
  val inicio: Timestamp? = null, val fim: Timestamp? = null)

class ReservaRepository @Inject constructor() {
  private val col = Firebase.firestore.collection("reservas")

  suspend fun criar(r: ReservaDoc): String =
    col.add(r).await().id

  // reemite a cada mudança no servidor ou no cache
  fun daBancada(id: String): Flow<List<ReservaDoc>> = col
    .whereEqualTo("bancadaId", id)
    .whereGreaterThan("fim", Timestamp.now())
    .orderBy("fim")
    .snapshots()
    .map { it.toObjects<ReservaDoc>() }

  suspend fun cancelar(docId: String) {
    col.document(docId).delete().await()
  }
}
```

`snapshots()`

Transforma o *listener* em tempo real do Firestore em `Flow`. Quando outra pessoa cria uma reserva, todas as telas abertas atualizam.

Offline

Cache local ativo: lê sem rede e envia as escritas quando a conexão volta.

Índices

Filtros em campos diferentes exigem índice composto (o Logcat traz o link).

Transações: evitando reservas duplicadas

Reserva atômica

```
suspend fun reservarHorario(b: String, slot: String,
                            uid: String) {
  val db = Firebase.firestore
  val ref = db.collection("bancadas").document(b)
    .collection("slots").document(slot)  // ex.: 2026-11-03T14

  db.runTransaction { tx ->
    val atual = tx.get(ref)
    if (atual.exists()) {
      throw FirebaseFirestoreException("Horário ocupado",
        FirebaseFirestoreException.Code.ABORTED)
    }
    tx.set(ref, mapOf("uid" to uid,
      "criadoEm" to FieldValue.serverTimestamp()))
  }.await()
}
```

Transação

Lê e grava como uma operação única. Se outro cliente alterar o documento no meio, o Firestore repete a transação, e duas pessoas nunca ficam com o mesmo horário.

Modelagem

Usar o horário como **ID do documento** torna o conflito impossível por construção: só pode existir um `slots/2026-11-03T14`.

Atenção

Transações precisam de conexão com o servidor; não funcionam offline.

Regras de segurança

Regras do Firestore

firestore.rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    match /bancadas/{id} {
      allow read: if request.auth != null;
      allow write: if request.auth.token.admin == true;
    }

    match /reservas/{rid} {
      allow read: if request.auth != null;
      allow create: if request.auth != null
        && request.resource.data.uid == request.auth.uid
        && request.resource.data.fim
             > request.resource.data.inicio;
      allow delete: if resource.data.uid == request.auth.uid;
    }
  }
}
```

O erro mais grave

Publicar com `allow read, write: if true;` (“modo de teste”). Qualquer pessoa na internet lê e apaga todo o banco.

Como pensar

O app **não é confiável**: as regras, que rodam no servidor, são a única barreira real.

Emulator Suite

`firebase emulators:start` roda Auth, Firestore e Storage localmente, sem custo e sem dados reais.

Storage, mensagens e monitoramento

Cloud Storage: fotos dos equipamentos

FotoRepository.kt

```
suspend fun enviarFoto(patrimonio: String, uri: Uri): String {
  val ref = Firebase.storage.reference
    .child("equipamentos/$patrimonio.jpg")
  ref.putFile(uri).await()                    // upload
  return ref.downloadUrl.await().toString()
}

// No ViewModel: grava a URL no documento do equipamento
val url = fotos.enviarFoto("IFMA-0042", uriEscolhida)
equipamentos.atualizarFoto("IFMA-0042", url)
```

Dica

Comprima a imagem antes de enviar (ex.: 1280 px, JPEG 80 %). Fotos de câmera têm vários MB.

storage.rules

```
match /equipamentos/{arquivo} {
  allow read: if request.auth != null;
  allow write: if request.auth != null
    && request.resource.size < 5 * 1024 * 1024
    && request.resource.contentType
         .matches('image/.*');
}
```

Conceito

Arquivos ficam no Storage; o Firestore guarda apenas a URL e os metadados.

Cloud Messaging: notificações push



AndroidManifest.xml

```
<uses-permission android:name=
  "android.permission.POST_NOTIFICATIONS"/>
```

MensagensService.kt

```
class MensagensService : FirebaseMessagingService() {
  override fun onNewToken(token: String) {
    // envie ao seu backend / Firestore
    salvarTokenDoUsuario(token)
  }
  override fun onMessageReceived(msg: RemoteMessage) {
    val titulo = msg.notification?.title ?:
      "LabMonitor"
    mostrarNotificacao(titulo, msg.data["texto"])
  }
}
```

Android 13+

Notificações exigem a permissão em tempo de execução `POST_NOTIFICATIONS` (Módulo 10). Registre o *service* no manifesto.

Crashlytics, Analytics e alternativas ao Firebase

Crashlytics

Agrupa as falhas de produção por causa, com pilha de chamadas, modelo do aparelho e versão do Android. Use `Firebase.crashlytics.log("...")` e `setCustomKey` para dar contexto.

Analytics

`logEvent("reserva_criada")` registra eventos e mede o uso real das funcionalidades.

LGPD

Informe na política de privacidade quais dados são coletados e por quê (Módulo 12).

Alternativa	Características
Supabase	PostgreSQL (SQL e relações), auth, storage, tempo real; código aberto
Appwrite	código aberto, pode ser hospedado no próprio servidor
Backend próprio	Ktor, Spring, FastAPI ou Node com banco relacional; controle total, mais trabalho

Mantenha a porta aberta

Com a arquitetura do Módulo 06, o Firebase fica atrás de interfaces de repositório. Trocar de backend afeta só a camada de dados.

Fechamento

Resumo

Resumo

- Firebase entrega autenticação, banco, arquivos, push e monitoramento sem servidor próprio.
- Configure com `google-services.json`, o plugin e o BoM; use `await()` e `snapshots()` com coroutines.
- Firestore: coleções e documentos, consultas com índices, cache offline e transações.
- As regras de segurança são a proteção real dos dados; teste-as no Emulator Suite.
- Storage guarda arquivos (exige plano Blaze); FCM entrega notificações push.
- Isole o Firebase atrás de repositórios para poder trocar de backend.

Exercícios

Exercício 1 — Autenticação

Implemente as telas de login, cadastro e "esqueci a senha" (`sendPasswordResetEmail`) com validação e mensagens de erro em português.

Exercício 2 — Regras

Escreva regras que permitam a qualquer usuário logado ler as bancadas, mas só o dono editar ou cancelar a própria reserva. Teste no emulador com dois usuários.

Exercício 3 — Tempo real

Abra o app em dois aparelhos com usuários diferentes. Crie uma reserva em um e observe o outro. Desligue a rede de um deles e explique o comportamento.

LabMonitor — Etapa 9

Adicione login, reservas de bancada no Firestore com transação anti-conflito e regras de segurança. Integre o Crashlytics.

Para aprofundar

- Firebase para Android: `https://firebase.google.com/docs/android/setup`
- Authentication: `https://firebase.google.com/docs/auth/android/start`
- Credential Manager: `https://developer.android.com/identity/sign-in/credential-manager`
- Cloud Firestore: `https://firebase.google.com/docs/firestore`
- Regras de segurança: `https://firebase.google.com/docs/rules`
- Emulator Suite: `https://firebase.google.com/docs/emulator-suite`
- Preços: `https://firebase.google.com/pricing`

Fim do Módulo 09

Próximo: Módulo 10 — Recursos do dispositivo e IoT: permissões, câmera, localização, WorkManager, sensores, BLE e MQTT com ESP32.